

LAPORAN PRATIKUM

STRUKTUR DATA

Double Linked List

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 12 Mei 2026

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 Double Linked List	2
2.2 Memasukkan Nilai untuk DLL	2
2.2.1 Memasukkan Node pada Posisi Pertama	2
2.2.2 Memasukkan Node pada Posisi Terakhir.....	3
2.2.3 Memasukkan Node pada Posisi Tertentu	3
2.2.4 Method untuk Menampilkan DLL dan Method Main Sebagai Pengujian	4
2.3 Penelusuran dalam DLL	5
2.3.1 Penelusuran Maju.....	5
2.3.2 Penelusuran Mundur	5
2.3.3 Pengujian Penelusuran	5
2.4 Penghapusan Node.....	6
2.4.1 Menghapus Head.....	6
2.4.2 Menghapus Node Terakhir.....	6
2.4.3 Menghapus Node Pada Posisi Tertentu.....	7
2.4.4 Menguji Penghapusan Node	7
BAB III KESIMPULAN.....	8
LAPORAN TUGAS PRAKTIKUM.....	9
L.1 Method-Method Bantuan.....	9
L.2 Fungsi TambahLagu	9
L.3 Method Menampilkan Lagu	10
L.4 Method Mencari Judul Lagu.....	11
L.5 Pengujian Program	11
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya telah dipelajari Single Linked List yang setiap node-nya hanya memiliki satu pointer yang menunjuk ke node berikutnya. Meskipun SLL sudah lebih fleksibel dibanding array, ia memiliki keterbatasan yaitu traversal hanya bisa dilakukan satu arah dari head ke tail. Hal ini menjadi tidak efisien ketika dibutuhkan akses ke node sebelumnya, karena harus menelusuri ulang dari head.

Untuk mengatasi keterbatasan tersebut, digunakan Double Linked List sebuah variasi Linked List di mana setiap node memiliki dua pointer, yaitu next yang menunjuk ke node berikutnya dan prev yang menunjuk ke node sebelumnya. Dengan adanya dua pointer ini, traversal dapat dilakukan dari dua arah, baik dari head ke tail maupun dari tail ke head, sehingga operasi seperti penghapusan dan penyisipan node menjadi lebih mudah karena tidak perlu melacak node sebelumnya secara manual.

1.2 Tujuan

- Memahami konsep Double Linked List dan perbedaannya dengan Single Linked List dari sisi struktur node dan arah traversal.
- Mampu mengimplementasikan operasi penambahan node di depan, di belakang, dan pada posisi tertentu dalam DLL.
- Mampu mengimplementasikan operasi penelusuran dua arah, pencarian, dan penghapusan node pada DLL.
- Memahami kelebihan dan keterbatasan Double Linked List dibandingkan Single Linked List dalam konteks penggunaan memori dan efisiensi operasi.

1.3 Manfaat Praktikum

- Praktikan mampu menerapkan konsep Double Linked List dalam menyelesaikan permasalahan pemrograman yang membutuhkan traversal dua arah dan manipulasi node yang lebih fleksibel.
- Praktikan memahami bagaimana penambahan pointer prev pada setiap node meningkatkan kemudahan operasi dibandingkan Single Linked List, sekaligus memahami konsekuensinya terhadap penggunaan memori.
- Praktikan memiliki pemahaman yang komprehensif mengenai berbagai jenis Linked List sebagai fondasi untuk mempelajari struktur data lanjutan seperti Tree dan Graph.

BAB II

PEMBAHASAN

2.1 Double Linked List

```
1 package pekan6_2511532016;
2
3 public class NodeDLL_2511532016 {
4     int data_2016;
5     NodeDLL_2511532016 next_2016;
6     NodeDLL_2511532016 prev_2016;
7
8     public NodeDLL_2511532016(int data_2016) {
9         this.data_2016 = data_2016;
10        this.next_2016 = null;
11        this.prev_2016 = null;
12    }
13 }
```

Double Linked List (DLL) adalah sebuah struktur data berbasis node yang memiliki pointer dua arah, yaitu ke depan dan ke belakang.

Kode program di atas terdiri dari dua bagian, yaitu untuk deklarasi dan konstruktor. Atribut yang terdapat pada DLL adalah data, yaitu nilai dari node tersebut; next sebagai pointer ke node selanjutnya, dan prev sebagai pointer ke node sebelumnya.

Konstrukturnya sederhana. Hanya membuat sebuah node dengan nilai “data”, kedua pointer bernilai null.

2.2 Memasukkan Nilai untuk DLL

2.2.1 Memasukkan Node pada Posisi Pertama

```
static NodeDLL_2511532016 insertBegin_2016(NodeDLL_2511532016 head_2016, int data_2016) {
    NodeDLL_2511532016 new_node_2016 = new NodeDLL_2511532016(data_2016);
    new_node_2016.next_2016 = head_2016;
    if(head_2016 != null) {
        head_2016.prev_2016 = new_node_2016;
    }
    return new_node_2016;
}
```

Node baru dengan nama new_node dibuat, dengan nilai sesuai dengan yang diberikan pada argumen fungsi. Pointer next dari node ini akan merujuk pada node head pada DLL. Jika head memiliki nilai, maka pointer prev dari head akan merujuk pada new_node. Node new_node otomatis menjadi head dari DLL.

2.2.2 Memasukkan Node pada Posisi Terakhir

```
public static NodeDLL_2511532016 insertEnd_2016(NodeDLL_2511532016 head_2016, int newData_2016) {
    NodeDLL_2511532016 newNode_2016 = new NodeDLL_2511532016(newData_2016);
    if(head_2016 == null) head_2016 = newNode_2016;
    else {
        NodeDLL_2511532016 curr_2016 = head_2016;
        while(curr_2016.next_2016 != null) {
            curr_2016 = curr_2016.next_2016;
        }
        curr_2016.next_2016 = newNode_2016;
        newNode_2016.prev_2016 = curr_2016;
    }
    return head_2016;
}
```

Fungsi ini mirip dengan fungsi sebelumnya. Dibuat sebuah node baru dengan nama newNode dengan nilai sesuai yang diberikan oleh argumen dari fungsi yang memanggil. Jika DLL tidak memiliki head, node baru ini akan menjadi head nya.

Jika memiliki head, fungsi ini akan melakukan *traverse* pada DLL dimulai dari head dengan menggunakan node baru yaitu curr. Ia akan berhenti sampai mendapatkan nilai node selanjutnya yang bernilai null. Jika ditemukan, maka pointer dari curr dan newNode akan merujuk satu sama lain.

2.2.3 Memasukkan Node pada Posisi Tertentu

```
public static NodeDLL_2511532016 insertAtPosition_2016(NodeDLL_2511532016 head_2016, int pos, int new_data) {
    NodeDLL_2511532016 new_node_2016 = new NodeDLL_2511532016(new_data);
    if(pos == 1){
        new_node_2016.next_2016 = head_2016;
        if(head_2016 != null) {
            head_2016.prev_2016 = new_node_2016;
        }
        head_2016 = new_node_2016;
        return head_2016;
    }

    NodeDLL_2511532016 curr_2016 = head_2016;
    for(int i_2016=1; i_2016<pos - 1 && curr_2016 != null; ++i_2016) {
        curr_2016 = curr_2016.next_2016;
    }
    if(curr_2016 == null) {
        System.out.println("Posisi tidak ada");
        return head_2016;
    }
    new_node_2016.prev_2016 = curr_2016;
    new_node_2016.next_2016 = curr_2016.next_2016;
    curr_2016.next_2016 = new_node_2016;
    if(new_node_2016.next_2016 != null) {
        new_node_2016.next_2016.prev_2016 = new_node_2016;
    }
    return head_2016;
}
```

Pada fungsi ini, awalnya akan dibuat sebuah node baru dengan nama new_node. Kemudian dilakukan pengecekan posisi yang diinginkan. Jika posisi yang diinginkan adalah posisi pertama, ini sama saja dengan meletakkan pada posisi head, dan new_node akan menjadi head yang baru. Jika posisi yang diinginkan adalah selain 1, akan dibuat sebuah node baru dengan nama curr. Ia akan melakukan traverse sampai ke posisi yang diinginkan, atau ketika bertemu node dengan nilai null. Jika null, tampilkan “Posisi tidak ada” ke layar. Jika tidak null, ia akan berhenti

pada satu node dari posisi yang ditentukan. Pointer prev dari new_node akan merujuk pada node curr, dan pointer next nya akan merujuk pada node selanjutnya dari node curr. Dapat dikatakan, fungsi ini menyisipkan node pada posisi tertentu tanpa merusak tautan dari DLL asilnya.

2.2.4 Method untuk Menampilkan DLL dan Method Main Sebagai Pengujian

```
public static void printList_2016(NodeDLL_2511532016 head_2016){
NodeDLL_2511532016 curr_2016 = head_2016;
while (curr_2016 != null) {
    System.out.print(curr_2016.data_2016 + " <-> ");
    curr_2016 = curr_2016.next_2016;
}
System.out.println();
}
public static void main(String[] args) {
    NodeDLL_2511532016 head_2016 = new NodeDLL_2511532016(2);
    head_2016.next_2016 = new NodeDLL_2511532016(3);
    head_2016.next_2016.prev_2016 = head_2016;
    head_2016.next_2016.next_2016 = new NodeDLL_2511532016(5);
    head_2016.next_2016.next_2016.prev_2016 = head_2016.next_2016;

    System.out.print("DLL awal: ");
    printList_2016(head_2016);
    head_2016 = insertBegin_2016(head_2016, 1);
    System.out.print("Simpul 6 ditambah di akhir: ");
    int data_2016 = 6;
    head_2016 = insertEnd_2016(head_2016, data_2016);
    printList_2016(head_2016);

    System.out.print("Tambah node pada posisi 4: ");
    int data2_2016 = 4;
    int pos_2016 =4;

    head_2016= insertAtPosition_2016(head_2016, pos_2016, data2_2016);
    printList_2016(head_2016);
}
```

Method printList digunakan untuk menampilkan isi dari DLL, dan tiap node-nya dipisahkan oleh tanda ↔.

Hasil dari method main ketika dijalankan adalah sebagai berikut.

```
DLL awal: 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node pada posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.3 Penelusuran dalam DLL

2.3.1 Penelusuran Maju

```
static void forwardTraversal_2016(NodeDLL_2511532016 head_2016) {
    NodeDLL_2511532016 curr_2016 = head_2016;
    while(curr_2016 != null){
        System.out.print(curr_2016.data_2016 + " <-> ");
        curr_2016 = curr_2016.next_2016;
    }
    System.out.println();
}
```

Method ini sama persis dengan method printList sebelumnya, yaitu menampilkan DLL yang dimulai dari head sampai node paling akhir

2.3.2 Penelusuran Mundur

```
static void backwardTraversal_2016(NodeDLL_2511532016 tail_2016) {
    NodeDLL_2511532016 curr_2016 = tail_2016;
    while(curr_2016 != null){
        System.out.print(curr_2016.data_2016 + " <-> ");
        curr_2016 = curr_2016.prev_2016;
    }
    System.out.println();
}
```

Method ini dapat dikatakan sebagai kebalikan dari method sebelumnya. Method ini menampilkan node dari DLL yang dimulai dari node paling akhir menuju ke node head.

2.3.3 Pengujian Penelusuran

```
public static void main(String[] args) {
    NodeDLL_2511532016 head_2016 = new NodeDLL_2511532016(1);
    NodeDLL_2511532016 second_2016 = new NodeDLL_2511532016(2);
    NodeDLL_2511532016 third_2016 = new NodeDLL_2511532016(3);

    head_2016.next_2016 = second_2016;
    second_2016.prev_2016 = head_2016;
    second_2016.next_2016 = third_2016;
    third_2016.prev_2016 = second_2016;

    System.out.println("Penelusuran maju: "); forwardTraversal_2016(head_2016);
    System.out.println("Penelusuran mundur: "); backwardTraversal_2016(third_2016);
}
```


Awalnya dibuat tiga buah node dengan nilai 1, 2, dan 3 berturut-turut. Kemudian, pointer mereka direfrenseikan ke node-node sebelumnya. Dapat divisualisasikan sebagai head↔second↔third↔. Keluaran dari program di atas adalah sebagai berikut.

```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur
3 <-> 2 <-> 1 <->
```

2.4 Penghapusan Node

2.4.1 Menghapus Head

```
public static NodeDLL_2511532016 delHead_2016(NodeDLL_2511532016 head_2016) {
    if(head_2016 == null) return null;
    head_2016 = head_2016.next_2016;
    if(head_2016 != null) head_2016.prev_2016 = null;
    return head_2016;
}
```

Tujuan fungsi ini adalah untuk menghapus head dari DLL. Jika head bernilai null, maka nilai kembaliannya adalah null. Jika tidak, head akan diubah menjadi node selanjutnya. Kemudian, head yang lama akan dihapus menggunakan head yang baru dengan pointer prev.

2.4.2 Menghapus Node Terakhir

```
public static NodeDLL_2511532016 dellast_2016(NodeDLL_2511532016 head_2016) {
    if(head_2016 == null || head_2016.next_2016 == null) return null;
    NodeDLL_2511532016 curr_2016 = head_2016;
    while(curr_2016.next_2016 != null) {
        curr_2016 = curr_2016.next_2016;
    }
    if(curr_2016.prev_2016 != null) {
        curr_2016.prev_2016.next_2016 = null;
    }
    return head_2016;
}
```

Fungsi ini bertujuan untuk menghapus node paling akhir pada DLL. Awalnya dilakukan pengecekan, yaitu jika head atau node selanjutnya dari head bernilai null, maka akan mengembalikan nilai null. Jika tidak, node baru dengan nama curr akan melakukan traverse sampai menemukan node yang node selanjutnya bernilai null. Jika sudah ditemukan, node terakhir akan dihapus menggunakan pointer dari node sebelumnya.

2.4.3 Menghapus Node Pada Posisi Tertentu

```
public static NodeDLL_2511532016 delPos_2016(NodeDLL_2511532016 head_2016, int pos_2016) {
    if(head_2016 == null) return head_2016;
    NodeDLL_2511532016 curr_2016 = head_2016;
    for(int i_2016=1; curr_2016 != null && i_2016 < pos_2016; ++i_2016) {
        curr_2016 = curr_2016.next_2016;
    }
    if(curr_2016 == null) return head_2016;
    if(curr_2016.prev_2016 != null) curr_2016.prev_2016.next_2016 = curr_2016.next_2016;
    if(curr_2016.next_2016 != null) curr_2016.next_2016.prev_2016 = curr_2016.prev_2016;
    if(head_2016 == curr_2016) head_2016 = curr_2016.next_2016;
    return head_2016;
}
```

Fungsi ini mirip dengan fungsi insertAtPosition, tetapi alih-alih menambahkan, ia berfungsi untuk menghapus node, dan menyambungkan node-node tetangganya agar DLL masih tetap berfungsi.

2.4.4 Menguji Penghapusan Node

```
public static void main(String[] args) {
    NodeDLL_2511532016 head_2016 = new NodeDLL_2511532016(1);
    head_2016.next_2016 = new NodeDLL_2511532016(2);
    head_2016.next_2016.prev_2016 = head_2016;
    head_2016.next_2016.next_2016 = new NodeDLL_2511532016(3);
    head_2016.next_2016.next_2016.prev_2016 = head_2016.next_2016;
    head_2016.next_2016.next_2016.next_2016 = new NodeDLL_2511532016(4);
    head_2016.next_2016.next_2016.next_2016.prev_2016 = head_2016.next_2016.next_2016;
    head_2016.next_2016.next_2016.next_2016.next_2016 = new NodeDLL_2511532016(5);
    head_2016.next_2016.next_2016.next_2016.next_2016.prev_2016 = head_2016.next_2016.next_2016.next_2016;

    System.out.print("DLL awal: ");
    printList_2016(head_2016);

    System.out.print("DLL setelah head dihapus: ");
    head_2016 = delHead_2016(head_2016);
    printList_2016(head_2016);

    System.out.print("DLL setelah node terakhir dihapus: ");
    head_2016 = dellast_2016(head_2016);
    printList_2016(head_2016);

    System.out.print("Menghapus node ke 2: ");
    head_2016 = delPos_2016(head_2016, 2);
    printList_2016(head_2016);
}
```

Awalnya, dibuat 5 node yang saling terhubung, yaitu $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$. Kemudian, head-nya dihapus, sehingga menyisakan $2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$. Dihapus pula node terakhir, kini menyisakan $2 \leftrightarrow 3 \leftrightarrow 4$. Terakhir, node pada posisi kedua dihapuskan, sehingga hanya menyisakan $2 \leftrightarrow 4$.

BAB III

KESIMPULAN

LAPORAN TUGAS PRAKTIKUM

L.1 Method-Method Bantuan

```
//Method alias untuk System.out.println() agar terlihat lebih rapi
static void Jump_2016() {
    System.out.println();
}
//Method untuk menampilkan menu yang akan dipanggil pada method main
static void tampilkanMenu_2016() {
    System.out.println("=== Playlist Musik NIM: 2511532016 ===");
    System.out.println("1. Tambah Lagu");
    System.out.println("2. Hapus Lagu Pertama");
    System.out.println("3. Lihat Playlist (Maju)");
    System.out.println("4. Lihat Playlist (Mundur)");
    System.out.println("5. Cari Lagu");
    System.out.println("6. Keluar Dari Menu");
}
```

Method Jump bertujuan untuk menjadi alias dari System.out.println. Method ini digunakan untuk membuat baris baru dan supaya program terlihat lebih rapi. Ada juga method tampilkan menu untuk menampilkan pilihan yang tersedia kepada pengguna.

L.2 Fungsi TambahLagu

```
//Fungsi untuk menambahkan lagu ke dalam DLL
static Lagu_2511532016 TambahLagu_2016(Lagu_2511532016 head_2016) {
    Lagu_2511532016 newSong_2016 = new Lagu_2511532016();
    System.out.print("Masukkan Judul Lagu: ");
    String judul_2016 = sc_2016.nextLine();
    newSong_2016.setJudul_2016(judul_2016);
    System.out.print("Masukkan Penyanyi Lagu: ");
    String penyanyi_2016 = sc_2016.nextLine();
    newSong_2016.setPenyanyi_2016(penyanyi_2016);
    if(head_2016 == null) head_2016 = newSong_2016;
    else {
        Lagu_2511532016 curr_2016 = head_2016;
        while(curr_2016.getNext_2016() != null) {
            curr_2016 = curr_2016.getNext_2016();
        }
        curr_2016.setNext_2016(newSong_2016);
        newSong_2016.setPrev_2016(curr_2016);
    }
    System.out.println("Lagu " + newSong_2016.getJudul_2016() +
        " oleh " + newSong_2016.getPenyanyi_2016()+
        " Berhasil Ditambahkan ke dalam Playlist!\n");
    return head_2016;
}
```

Fungsi ini menerima input berupa judul lagu dan penyanyinya dari pengguna, kemudian menambahkannya pada DLL pada urutan paling belakang.

```
//Fungsi untuk menghapus lagu pertama (head) pada DLL
static Lagu_2511532016 HapusLaguPertama_2016(Lagu_2511532016 head_2016) {
    if(head_2016 == null) {
        System.out.println("Playlist Kosong!\n");
        return null;
    }
    System.out.println("Lagu " + head_2016.getJudul_2016() +
        " oleh " + head_2016.getPenyanyi_2016()+
        " Berhasil Dihapus!\n");
    head_2016 = head_2016.getNext_2016();
    if(head_2016 != null) {
        head_2016.setPrev_2016(null);
    }
    return head_2016;
}
```

Fungsi ini menghapus lagu pertama dalam DLL dan menampilkan namanya ketika berhasil dihapus.

L.3 Method Menampilkan Lagu

```
//Method untuk menampilkan daftar lagu dari kiri ke kanan (berdasarkan waktu dimasukkan)
static void ShowListFromLeft_2016(Lagu_2511532016 head_2016) {
    Lagu_2511532016 curr_2016 = head_2016;
    if(curr_2016 == null) {
        System.out.println("Playlist Kosong!\n");
        return;
    }
    while(curr_2016 != null) {
        System.out.println(curr_2016.getJudul_2016() + " - " + curr_2016.getPenyanyi_2016());
        curr_2016 = curr_2016.getNext_2016();
    }
    Jump_2016();
}
```

Method yang menampilkan daftar dari yang pertama masuk hingga terakhir masuk.

```
//Method untuk menampilkan daftar lagu dari kanan ke kiri (berdasarkan waktu dimasukkan)
static void ShowListFromRight(Lagu_2511532016 tail_2016) {
    Lagu_2511532016 curr_2016 = tail_2016;
    if(curr_2016 == null) {
        System.out.println("Playlist Kosong!\n");
        return;
    }
    //Pergi ke bagian ujung dulu
    while(curr_2016.getNext_2016() != null) {
        curr_2016 = curr_2016.getNext_2016();
    }
    while(curr_2016 != null) {
        System.out.println(curr_2016.getJudul_2016() + " - " + curr_2016.getPenyanyi_2016());
        curr_2016 = curr_2016.getPrev_2016();
    }
    Jump_2016();
}
```

Method yang menampilkan daftar dari yang terakhir masuk hingga pertama masuk.

L.4 Method Mencari Judul Lagu

```
//Method untuk mencari judul lagu
static void CariJudul_2016(Lagu_2511532016 head_2016) {
    if(head_2016 == null) {
        System.out.println("Playlist Kosong!\n");
        return;
    }
    Lagu_2511532016 curr_2016 = head_2016;
    System.out.print("Masukkan Judul Lagu yang Ingin Dicari: ");
    String judul_2016 = sc_2016.nextLine();
    while(curr_2016 != null) {
        if(curr_2016.judul_2016.toLowerCase().equals(judul_2016.toLowerCase())) {
            System.out.println("Lagu " + curr_2016.getJudul_2016() + " Terdapat dalam Playlist!\n");
            return;
        }
        curr_2016 = curr_2016.getNext_2016();
    }
    System.out.println("Lagu " + judul_2016 + " Tidak Terdapat dalam Playlist!\n");
    return;
}
```

Method ini bertujuan untuk mencari judul lagu yang terdapat dalam DLL. Lagu yang ingin dicari judulnya dikonversi menjadi huruf non-kapital agar memudahkan pengguna dalam mencari lagu yang diinginkan.

L.5 Pengujian Program

```
=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 1
Masukkan Judul Lagu: Beibeh
Masukkan Penyanyi Lagu: Jastin Bibia
Lagu Beibeh oleh Jastin Bibia Berhasil Ditambahkan ke dalam Playlist!

=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 1
Masukkan Judul Lagu: My Balon Gw
Masukkan Penyanyi Lagu: A.T. Mahmud
Lagu My Balon Gw oleh A.T. Mahmud Berhasil Ditambahkan ke dalam Playlist!

=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 5
Masukkan Judul Lagu yang Ingin Dicari: beibeh
Lagu Beibeh Terdapat dalam Playlist!
```

```

Pilih menu: 3
Beibeh - Jastin Bibia
My Balon Gw - A.T. Mahmud

=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 4
My Balon Gw - A.T. Mahmud
Beibeh - Jastin Bibia

=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 2
Lagu Beibeh oleh Jastin Bibia Berhasil Dihapus!

=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 3
My Balon Gw - A.T. Mahmud

=== Playlist Musik NIM: 2511532016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar Dari Menu
Pilih menu: 6
Program Selesai.

```

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>